

User Story 1 – Indexing and Query Optimization

1. Check Query Performance Before Index

use BookVerseDB

```
db.books.find({
  genre: "Fantasy"
}).explain("executionStats")
```

2. Create Indexes

Genre Index

```
db.books.createIndex({
  genre: 1
})
```

Author Index

```
db.books.createIndex({
  authorId: 1
})
```

Ratings Score Index

```
db.books.createIndex({
  "ratings.score": 1
})
```

View indexes:

```
db.books.getIndexes()
```

3. Compare Query Performance After Index

```
db.books.find({
  genre: "Fantasy"
}).explain("executionStats")
```

4. Drop Unnecessary Index

View indexes:

```
db.books.getIndexes()
```

Drop ratings index:

```
db.books.dropIndex("ratings.score_1")
```

or

```
db.books.dropIndex({
  "ratings.score": 1
})
```

Verify:

```
db.books.getIndexes()
```

User Story 2 – Aggregation Framework

1. Average Rating Per Book

```
db.books.aggregate([
  {
    $unwind: "$ratings"
  },
  {
    $group: {
      _id: "$title",
      averageRating: {
        $avg: "$ratings.score"
      }
    }
  }
])
```

Output

```
[
  {
    "_id": "Harry Potter",
    "averageRating": 4.5
  },
  {
    "_id": "Foundation",
    "averageRating": 5
  }
]
```

2. Top 3 Highest Rated Books

```
db.books.aggregate([
  {
    $unwind: "$ratings"
  },
  {
    $group: {
      _id: "$title",
      avgRating: {
        $avg: "$ratings.score"
      }
    }
  },
  {
    $sort: {
      avgRating: -1
    }
  },
  {
    $limit: 3
  }
])
```

3. Number of Books Published Per Genre

```
db.books.aggregate([
{
  $group: {
    _id: "$genre",
    totalBooks: {
      $sum: 1
    }
  }
}
])
```

Output

```
[
{
  "_id": "Fantasy",
  "totalBooks": 3
},
{
  "_id": "Science Fiction",
  "totalBooks": 2
}
]
```

4. Authors With More Than 2 Books

```
db.books.aggregate([
{
  $group: {
    _id: "$authorId",
    totalBooks: {
      $sum: 1
    }
  }
},
{
  $match: {
    totalBooks: {
      $gt: 2
    }
  }
}
])
```

With sample data:

```
[
{
  "_id": 3,
  "totalBooks": 3
}
```

```
}  
]
```

5. Total Reward Points For Each Author

Assume:

Reward Points = Sum of all ratings

```
db.books.aggregate([  
  {  
    $unwind: "$ratings"  
  },  
  {  
    $group: {  
      _id: "$authorId",  
      rewardPoints: {  
        $sum: "$ratings.score"  
      }  
    }  
  }  
])
```

Author Name + Reward Points

```
db.books.aggregate([  
  {  
    $unwind: "$ratings"  
  },  
  {  
    $group: {  
      _id: "$authorId",  
      rewardPoints: {  
        $sum: "$ratings.score"  
      }  
    }  
  },  
  {  
    $lookup: {  
      from: "authors",  
      localField: "_id",  
      foreignField: "_id",  
      as: "author"  
    }  
  },  
  {  
    $unwind: "$author"  
  },  
  {  
    $project: {  
      _id: 0,  
      authorName: "$author.name",  
      rewardPoints: 1  
    }  
  }  
])
```

```
    rewardPoints: 1
  }
}
])
```

User Story 3 – MongoDB Atlas Connection

Get Connection String

Atlas → Connect → Drivers

Example:

mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/BookVerseCloudDB

Node.js Connection

Install mongoose:

npm install mongoose

Create app.js

```
const mongoose = require("mongoose");
```

```
mongoose.connect(
  "mongodb+srv://username:password@cluster0.xxxxx.mongodb.net/BookVerseCloudDB"
)
.then(() => {
  console.log("Atlas Connected");
})
.catch(err => {
  console.log(err);
});
```

Run:

node app.js